

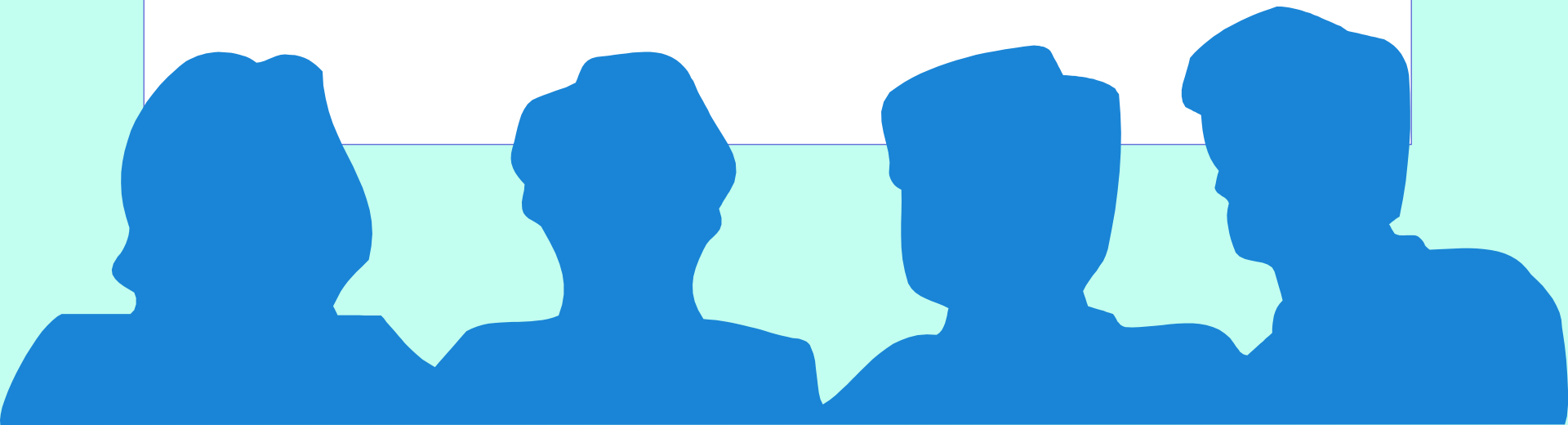
Computer Vision

Dr.Eng. Mahmoud Abu_Alfutuh

Chapter 3

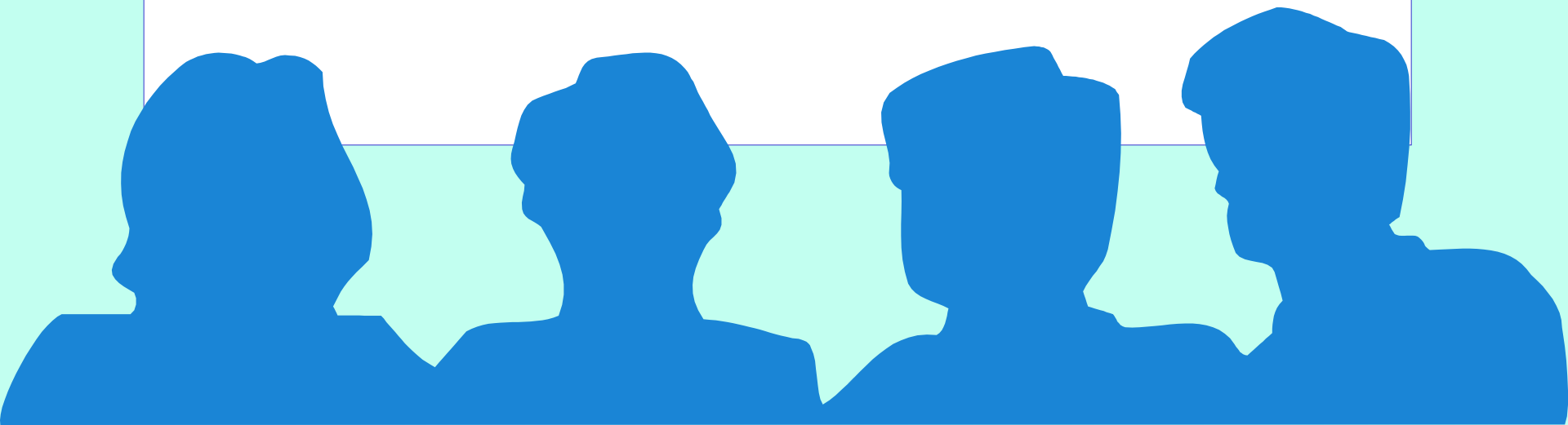
Image Preprocessing

Dr.Eng. Mahmoud Abu_Alfutuh



This chapter includes:

- 1. Introduction**
- 2. Filtering the image**
- 3. Intensity Transformation Functions**
- 4. Histogram transformation**



Overview

Intensity transformation functions

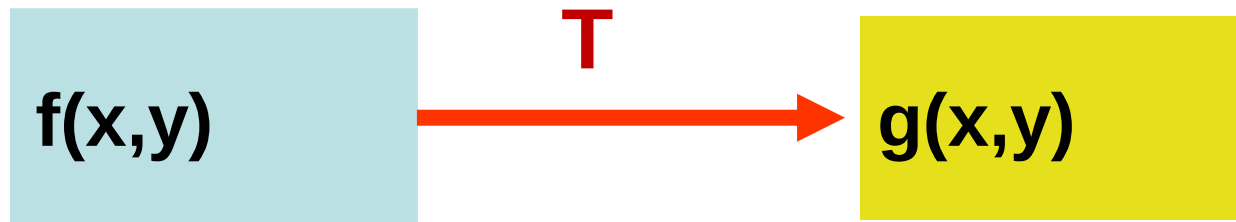
Gamma correction

Contrast stretching functions

Overview

T transforms the given image $f(x,y)$ into another image $g(x,y)$

T is called a gray-level or intensity transformation function



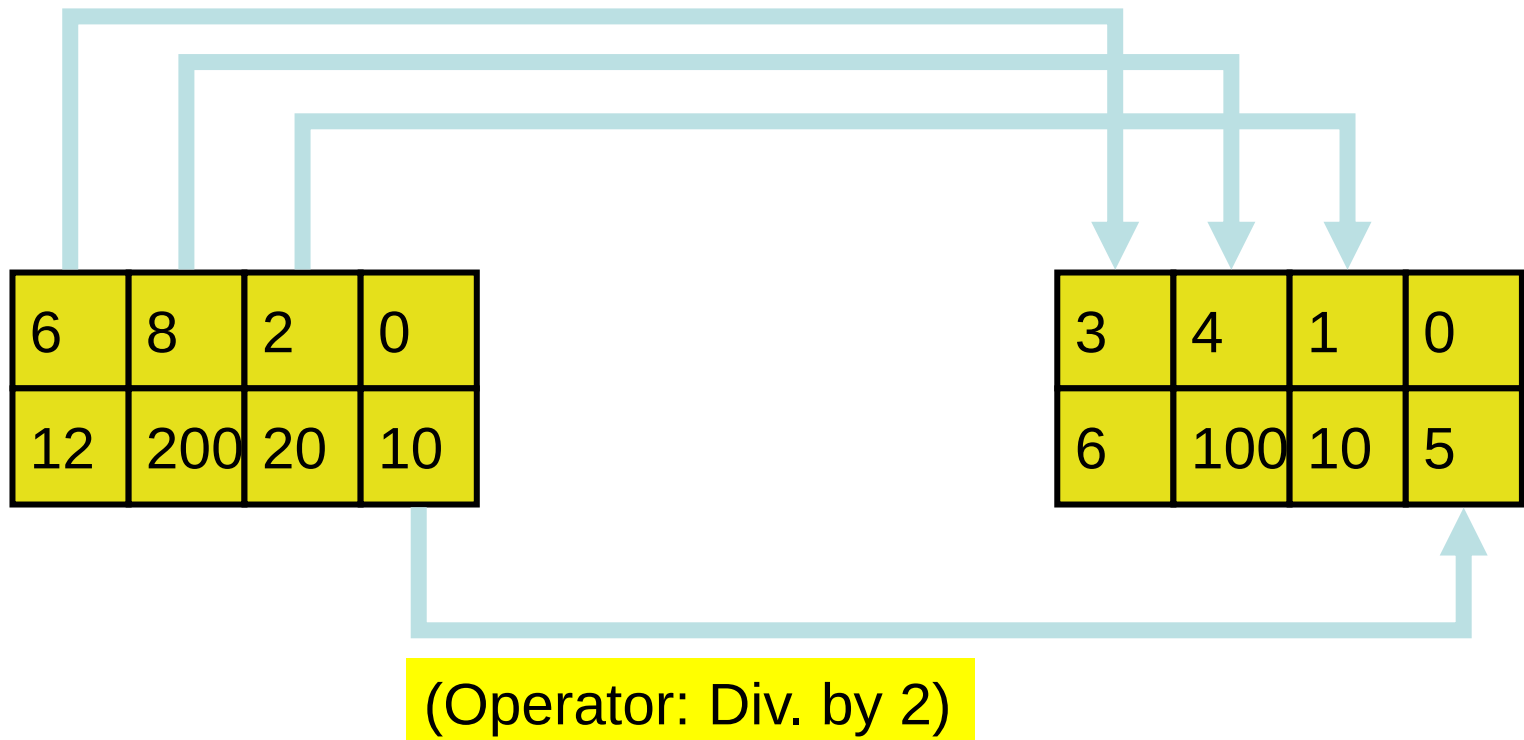
Spatial Domain

The operator T can be defined over:

- **The set of pixels (x,y) of the image**
- **The set of 'neighborhoods' $N(x,y)$ of each pixel**
- **A set of images $f_1, f_2, f_3, \dots$**

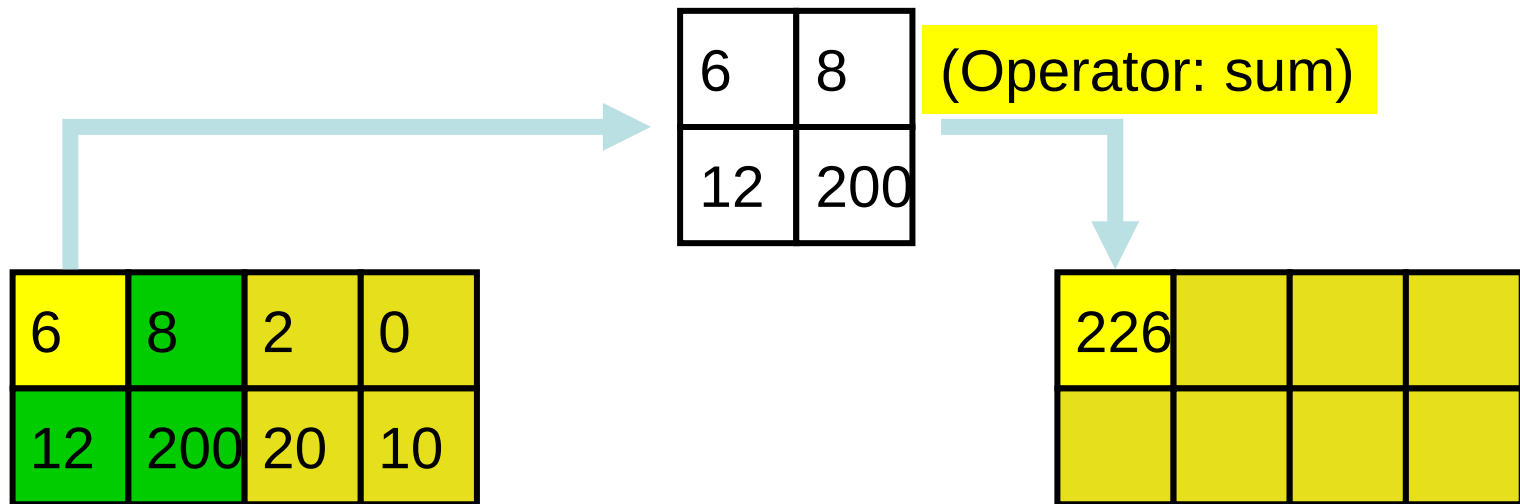
Spatial Domain

Operation on the set of image-pixels



Spatial Domain

Operation on the set of
'neighbourhoods' $N(x,y)$ of each pixel



Spatial Domain

Operation on a set of images $f_1, f_2, \dots$

6	8	2	0
12	200	20	10

(Operator: sum)

11	13	3	0
14	220	23	14

5	5	1	0
2	20	3	4

Intensity transformation functions

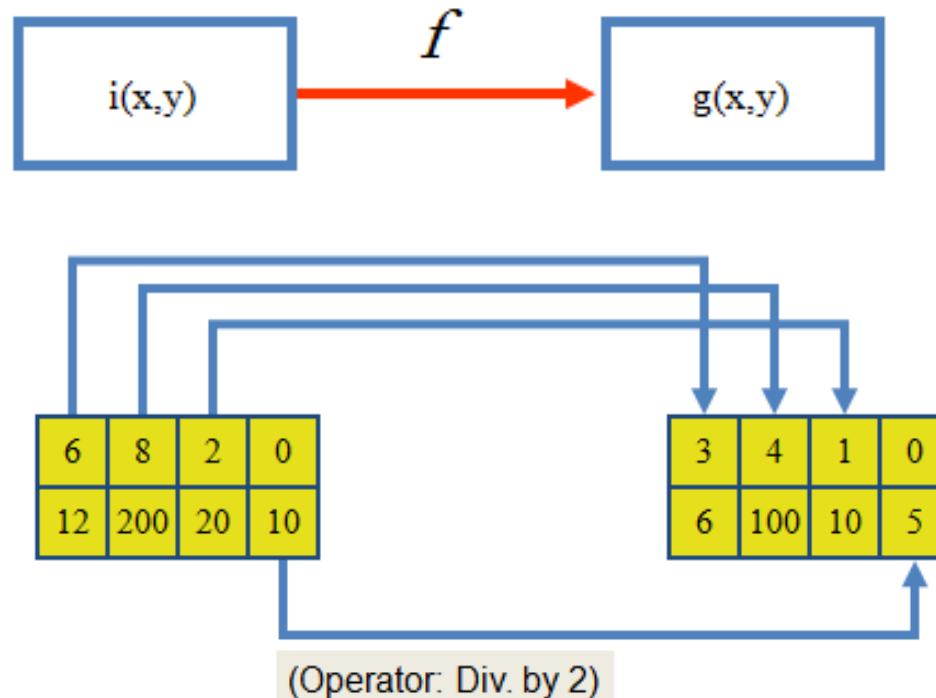
- An **intensity transformation function** is a function ***f*** applied to the intensity of each pixel:

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \\ p_{41} & p_{42} & p_{43} & p_{44} \end{bmatrix} \xrightarrow{f} \begin{bmatrix} f(p_{11}) & f(p_{12}) & f(p_{13}) & f(p_{14}) \\ f(p_{21}) & f(p_{22}) & f(p_{23}) & f(p_{24}) \\ f(p_{31}) & f(p_{32}) & f(p_{33}) & f(p_{34}) \\ f(p_{41}) & f(p_{42}) & f(p_{43}) & f(p_{44}) \end{bmatrix}$$

- Notice that here each pixel is processed by ***f*** individually.

Intensity transformation functions

- intensity transformation function f transforms the given image $i(x, y)$ into another image $g(x, y)$,
- where $g(x, y) = f(i(x, y))$



Intensity transformation functions

Example 1:

$$f(x) = 0$$



Intensity transformation functions

Example 2:

$$f(x) = x / 3$$



Intensity transformation functions

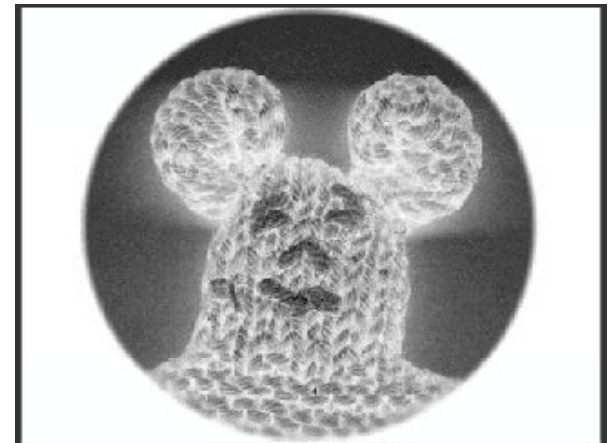
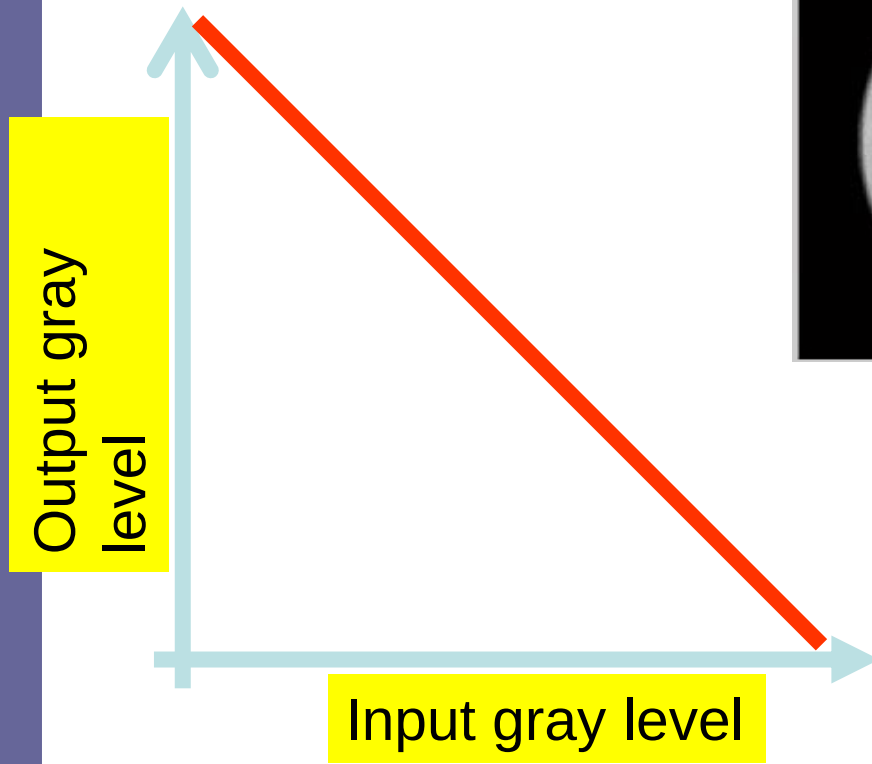
Basic Gray Level Transformations:

- Image Negatives
- Log Transformations
- Piecewise-Linear Transformation Functions

For the following slides L denotes the max. possible gray value of the image, i.e. $f(x,y) \in [0,L]$

Transformations

Image Negatives: $T(x) = L - x$



Intensity transformation functions

Image Negatives

Example 3:

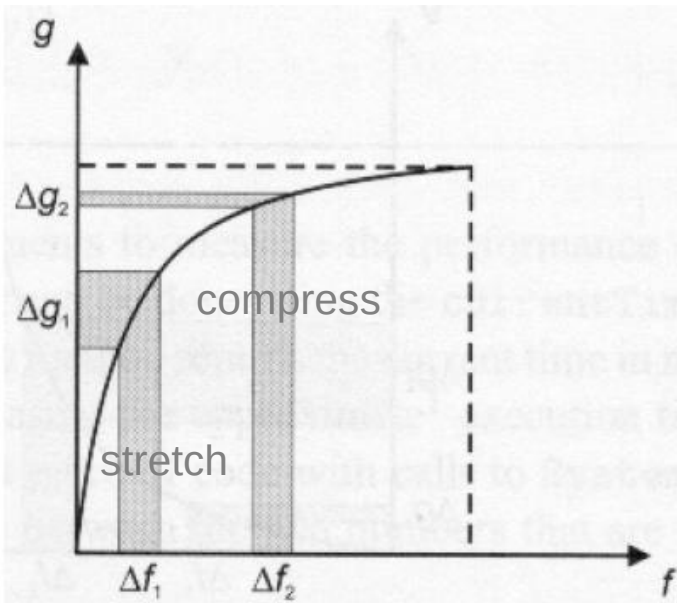
$$f(x) = 1 - x$$



Logarithmic transformation

- Enhance details in the darker regions of an image at the expense of detail in brighter regions.

$$g(f) = c * \log(1 + f)$$



Log Transformations



Log



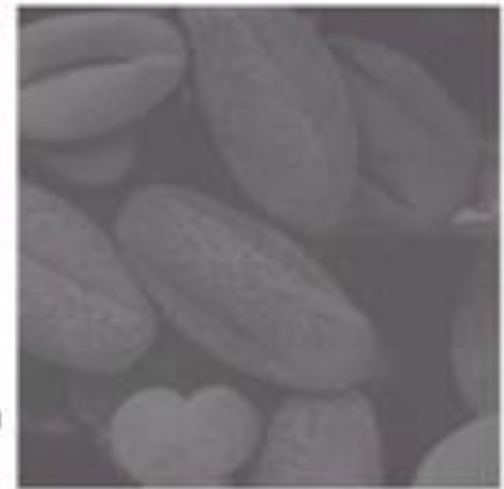
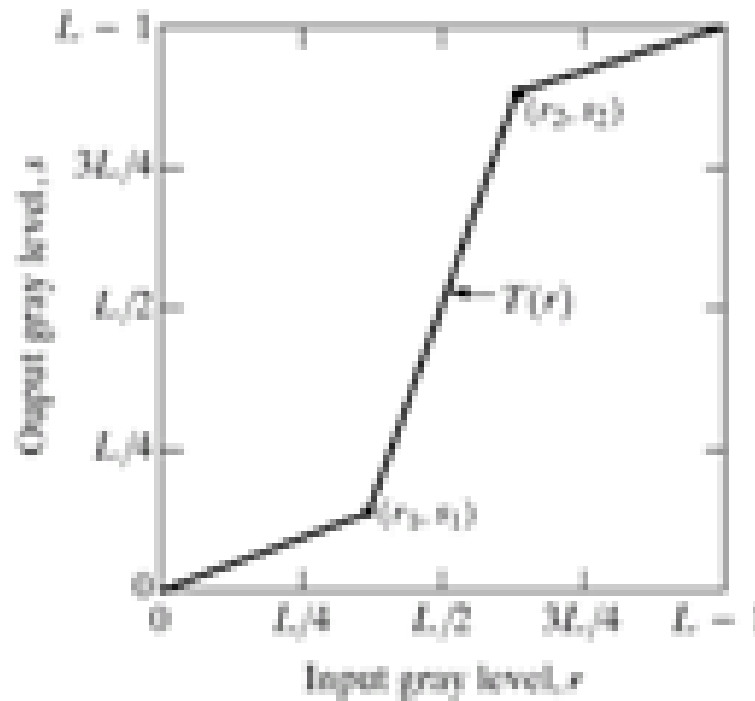
Intensity transformation functions

Example 4: Represents a **Piecewise** Linear Transformations:

$$f(x) = \begin{cases} 0 & \text{if } x < 1/4 \\ x & \text{if } 1/4 < x < 3/4 \\ 1 & \text{if } 3/4 < x \end{cases}$$



Piecewise Linear Transformations

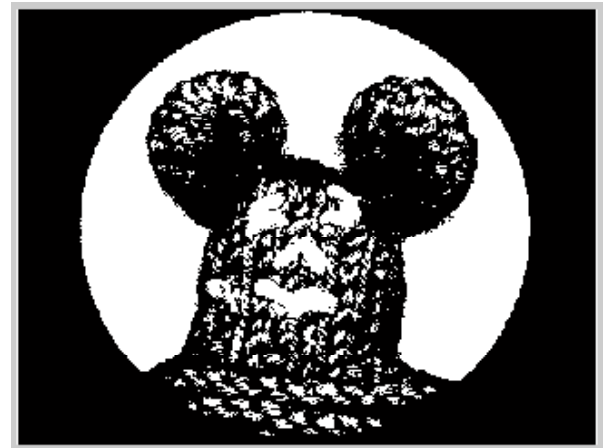
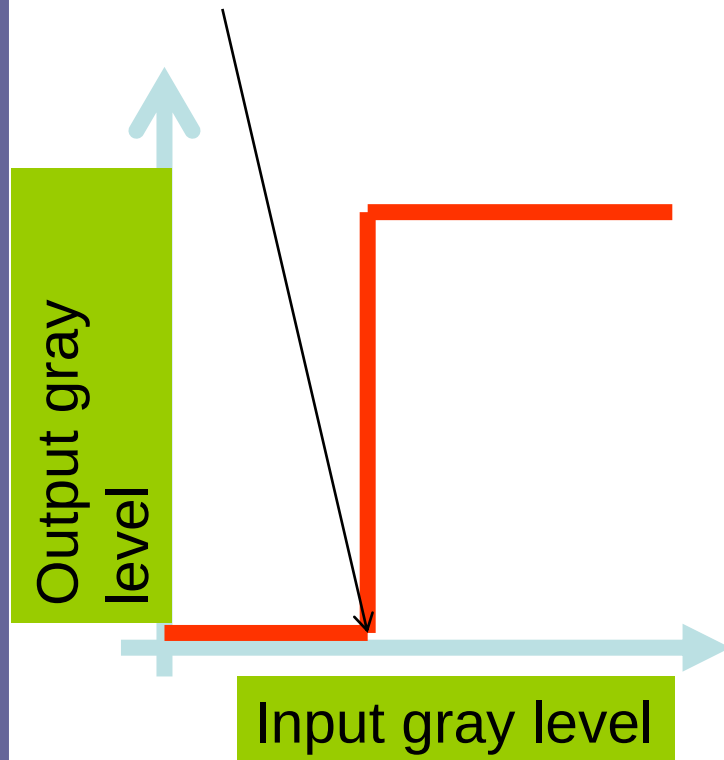


Piecewise Linear Transformations

Thresholding Function

$$g(x,y) = \begin{cases} L & \text{if } f(x,y) > t, \\ 0 & \text{else} \end{cases}$$

t = 'threshold level'



Piecewise Linear Transformations

Gray Level Slicing

Purpose: Highlight a specific range of gray values

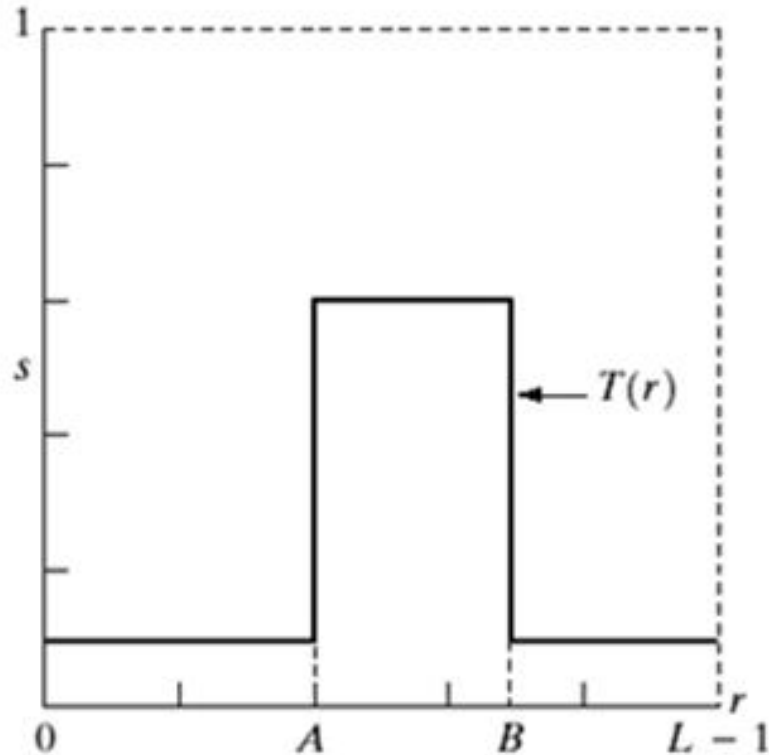
Two approaches:

1-Display high value for range of interest, low value else

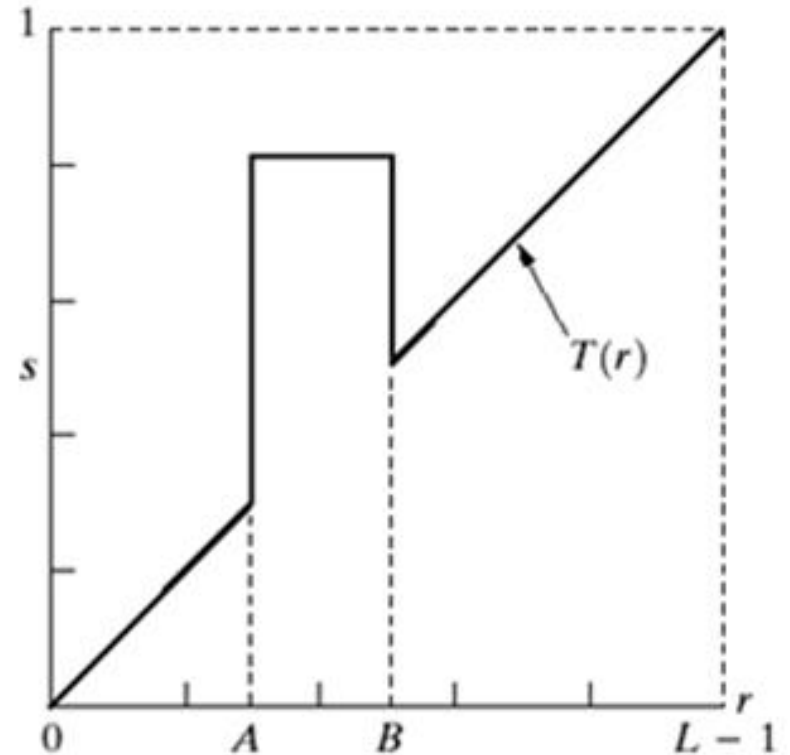
2-Display high value for range of interest, original value else

Piecewise Linear Transformations

Level Slicing



This transformation highlights range $[A, B]$ of gray levels and reduces all others to a constant level.



(b) This transformation highlights range $[A, B]$ but preserves all other levels.

Overview

Intensity transformation functions

Gamma correction

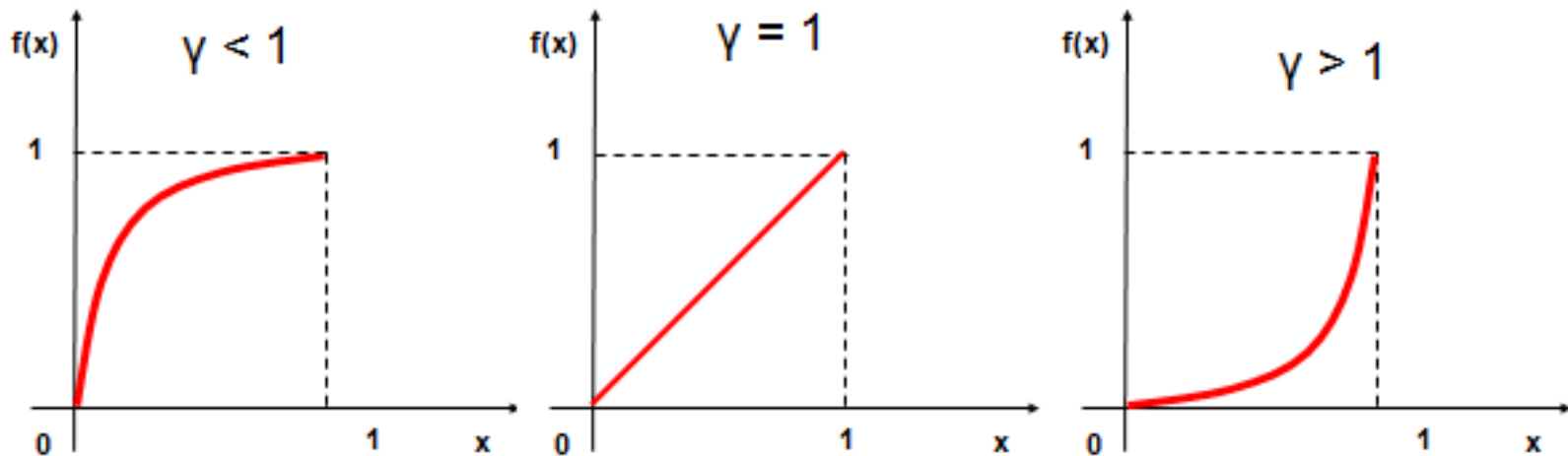
Contrast stretching functions

Gamma correction

- The gamma correction is an intensity transformation function in the form

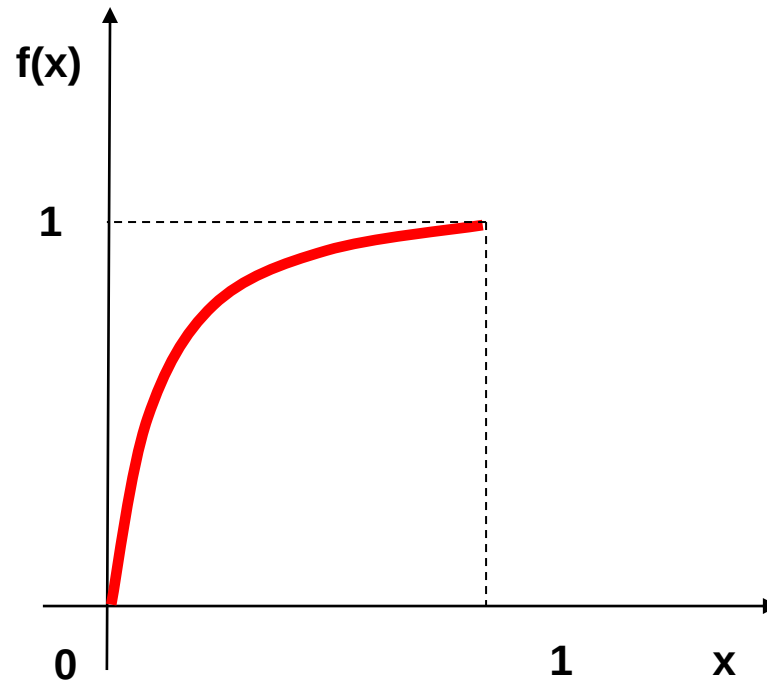
$$f(x) = x^\gamma$$

- where γ is a constant.



Gamma correction $f(x) = x^\gamma$

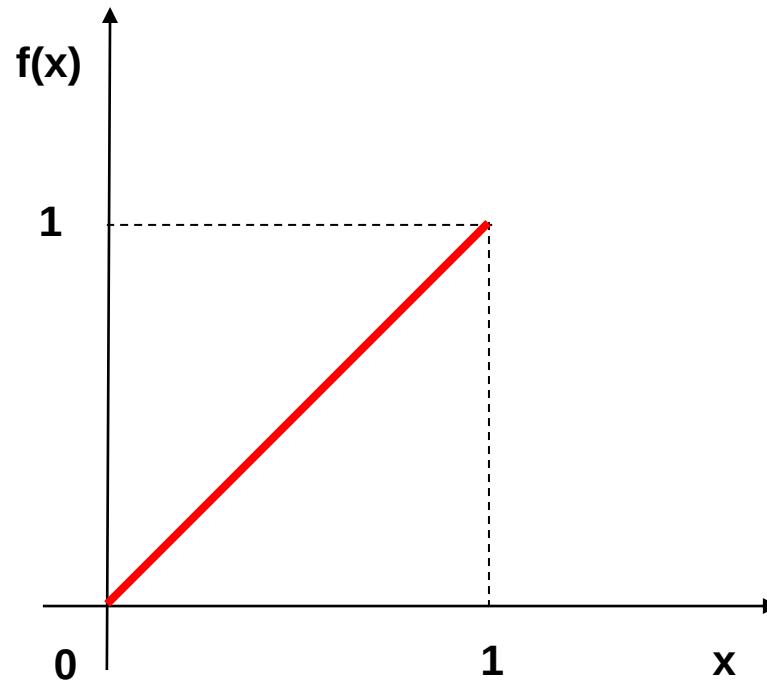
The graph of the gamma correction for $\gamma < 1$



Gamma correction

$$f(x) = x^\gamma$$

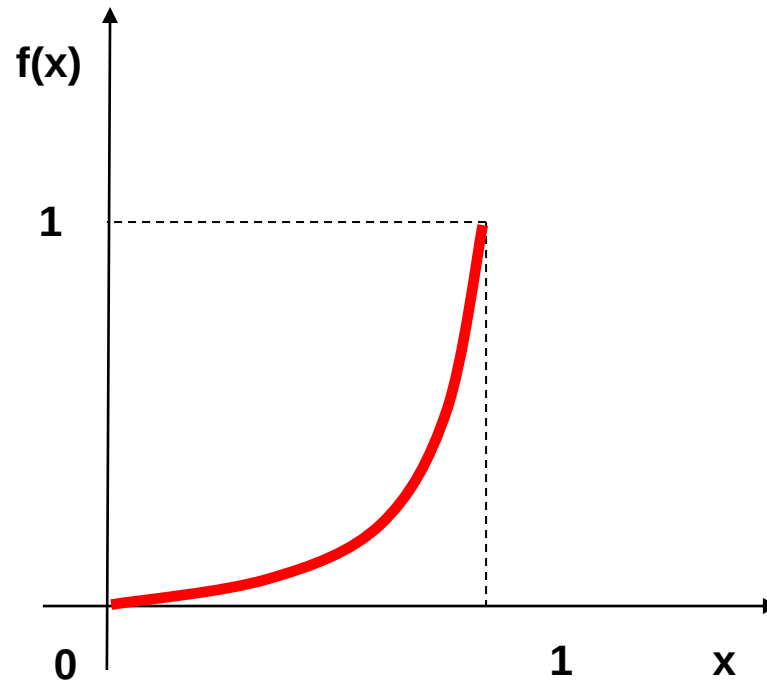
The graph of the gamma correction for $\gamma = 1$



Gamma correction

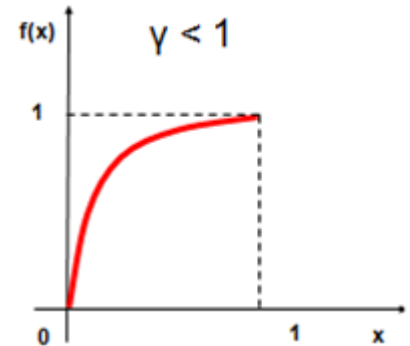
$$f(x) = x^\gamma$$

The graph of the gamma correction for $\gamma > 1$

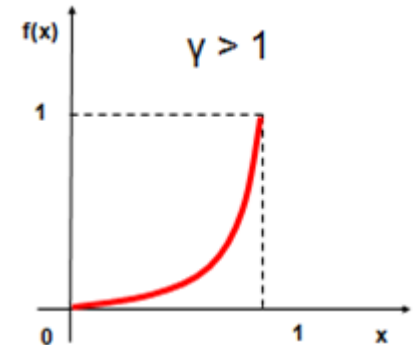


Gamma correction

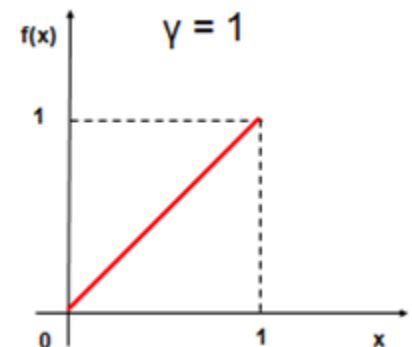
If $\gamma < 1$ the image is weighted toward higher (brighter) output values.



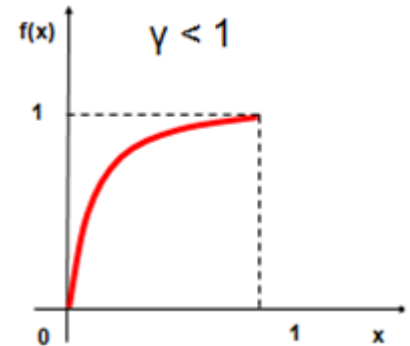
If $\gamma > 1$ the image is weighted toward lower (darker) output values.



If $\gamma = 1$ the transformation has no effect on the image.



Gamma correction: $\gamma < 1$



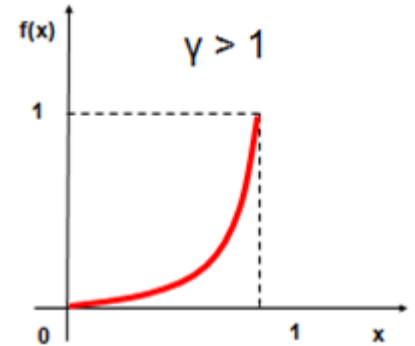
Example 5:

$$f(x) = x^{1/4}$$



If $\gamma < 1$ the image is weighted toward higher (brighter) output values.

Gamma correction: $\gamma > 1$



Example 6:

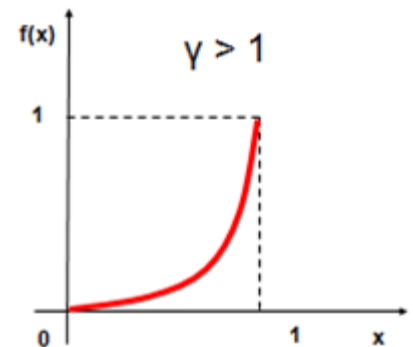
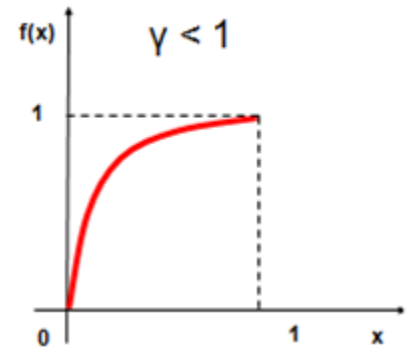
$$f(x) = x^2$$



If $\gamma > 1$ the image is weighted toward lower (darker) output values.

Gamma correction

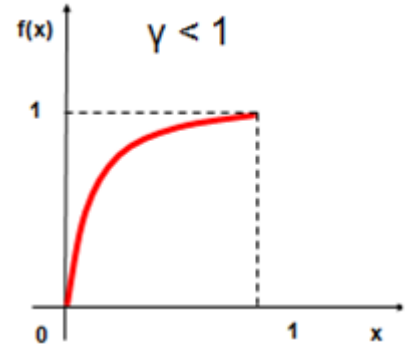
- The gamma correction is used to correct the tonality of an image.
- An underexposed photograph can be corrected using gamma correction with $\gamma < 1$.
- An overexposed photograph can be corrected using gamma correction with $\gamma > 1$.



Gamma correction

Example 7:

$$f(x) = x^\gamma$$



$\gamma = 1$



$\gamma = 1/2$



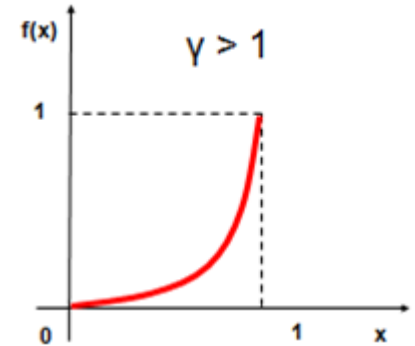
$\gamma = 1/3$



$\gamma = 1/4$

If $\gamma < 1$ the image is weighted toward higher (brighter) output values.

Gamma correction

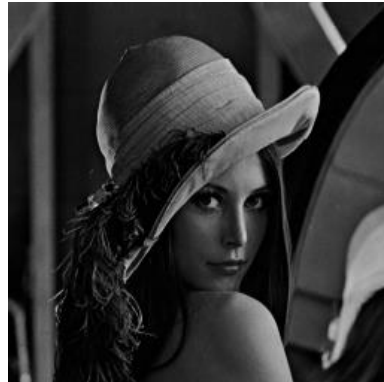


Example 2.7:

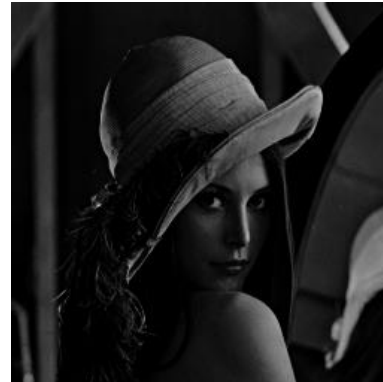
$$f(x) = x^\gamma$$



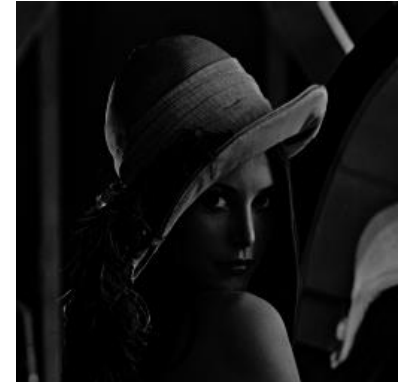
$\gamma = 1$



$\gamma = 2$



$\gamma = 3$



$\gamma = 4$

If $\gamma > 1$ the image is weighted toward lower (darker) output values.

Overview

Intensity transformation functions

Gamma correction

Contrast stretching functions

Contrast-stretching transformations

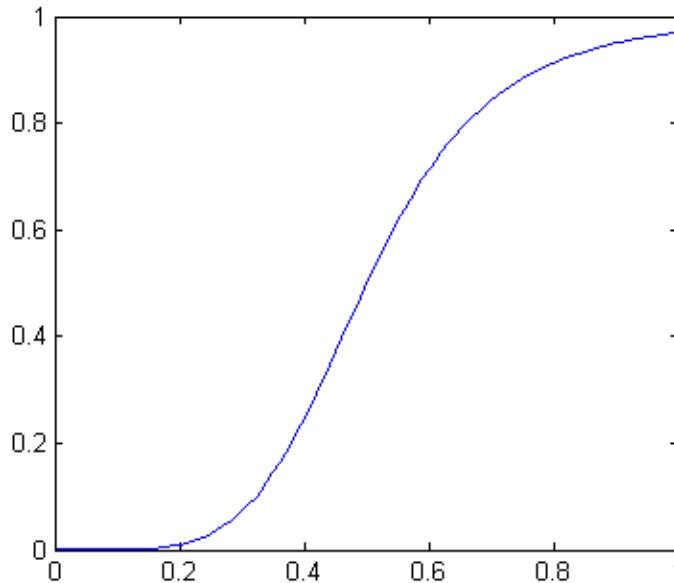
- The contrast stretching function

$$f(x) = \frac{1}{1 + (0.5/x)^E}$$

- where **E** is a relatively large constant (e.g. $E > 1$)
- increases the contrast of the image.

Contrast-stretching transformations

- The graph of the contrast stretching function for $E = 5$



$$f(x) = \frac{1}{1 + (0.5/x)^5}$$

- Pixel values below 0.5 are pushed nearer to 0.
- Pixel values above 0.5 are pushed nearer to 1.

Contrast-stretching transformations

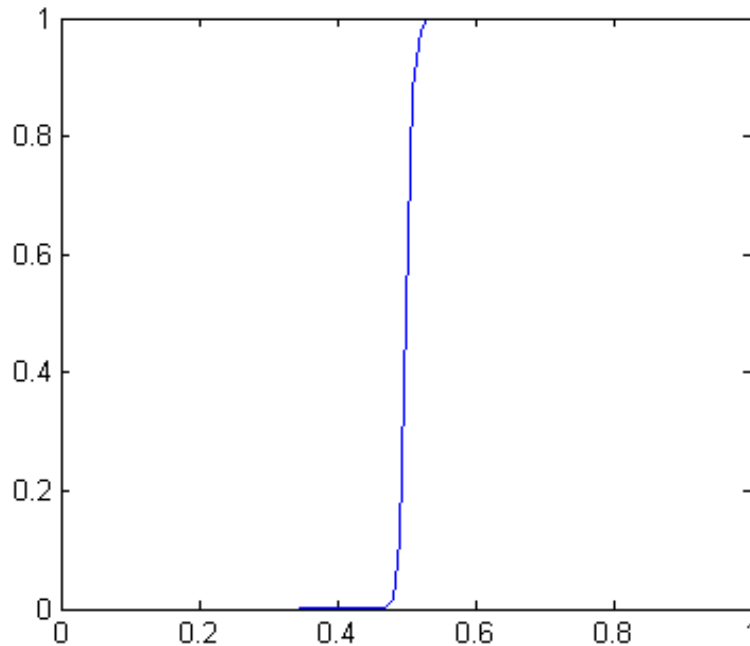
Example 8:

$$f(x) = \frac{1}{1 + (0.5/x)^5}$$



Contrast-stretching transformations

The graph of the contrast stretching function for $E = 100$



$$f(x) = \frac{1}{1 + (0.5/x)^{100}}$$

Contrast-stretching transformations

Example 2.9:

$$f(x) = \frac{1}{1 + (0.5/x)^{100}}$$



Contrast-stretching transformations

- Example 10: The higher the values of E the greater the increase of the contrast.

$$f(x) = \frac{1}{1 + (0.5/x)^E}$$



Original



$E = 2$



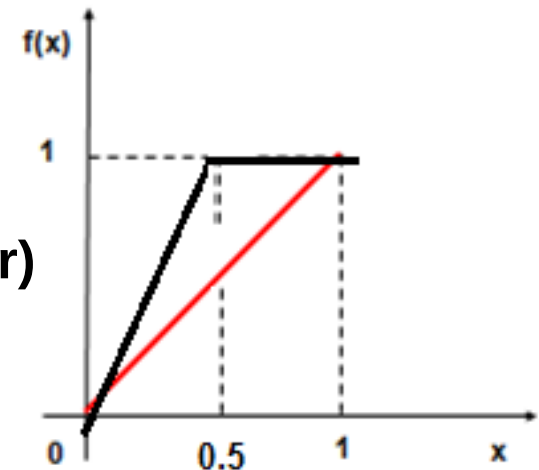
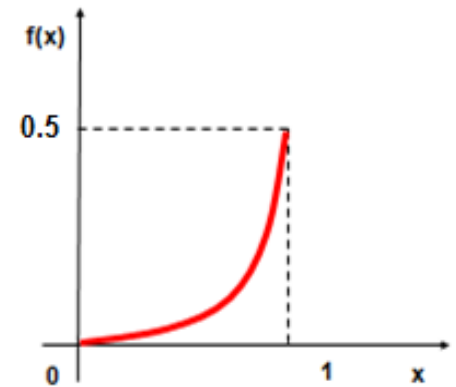
$E = 10$



$E = 50$

Example

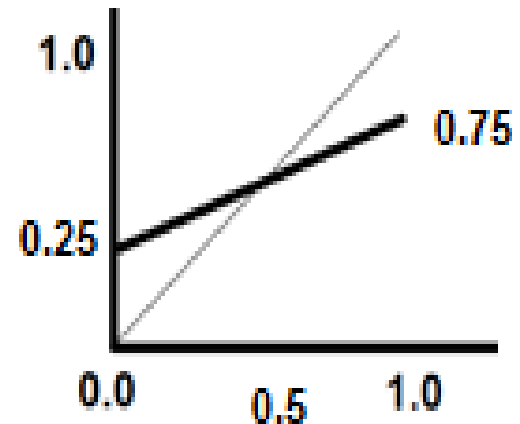
- Draw the graphs of the following intensity transformation functions, and then describe their effects on an image:
- (1) $f_1(x) = 0.5x^2$
- The image is weighted toward lower (darker) output values.
- (2) $f_2(x) = 2x$
- The image is weighted toward higher (brighter) output values.



Example

- Draw the graphs of the following intensity transformation functions, and then describe their effects on an image:

$$f(X) = 0.5X + 0.25$$



- Pixel values below 0.5 are weighted toward higher (brighter) output values .
- Pixel values above 0.5 are weighted toward lower (darker) output values.

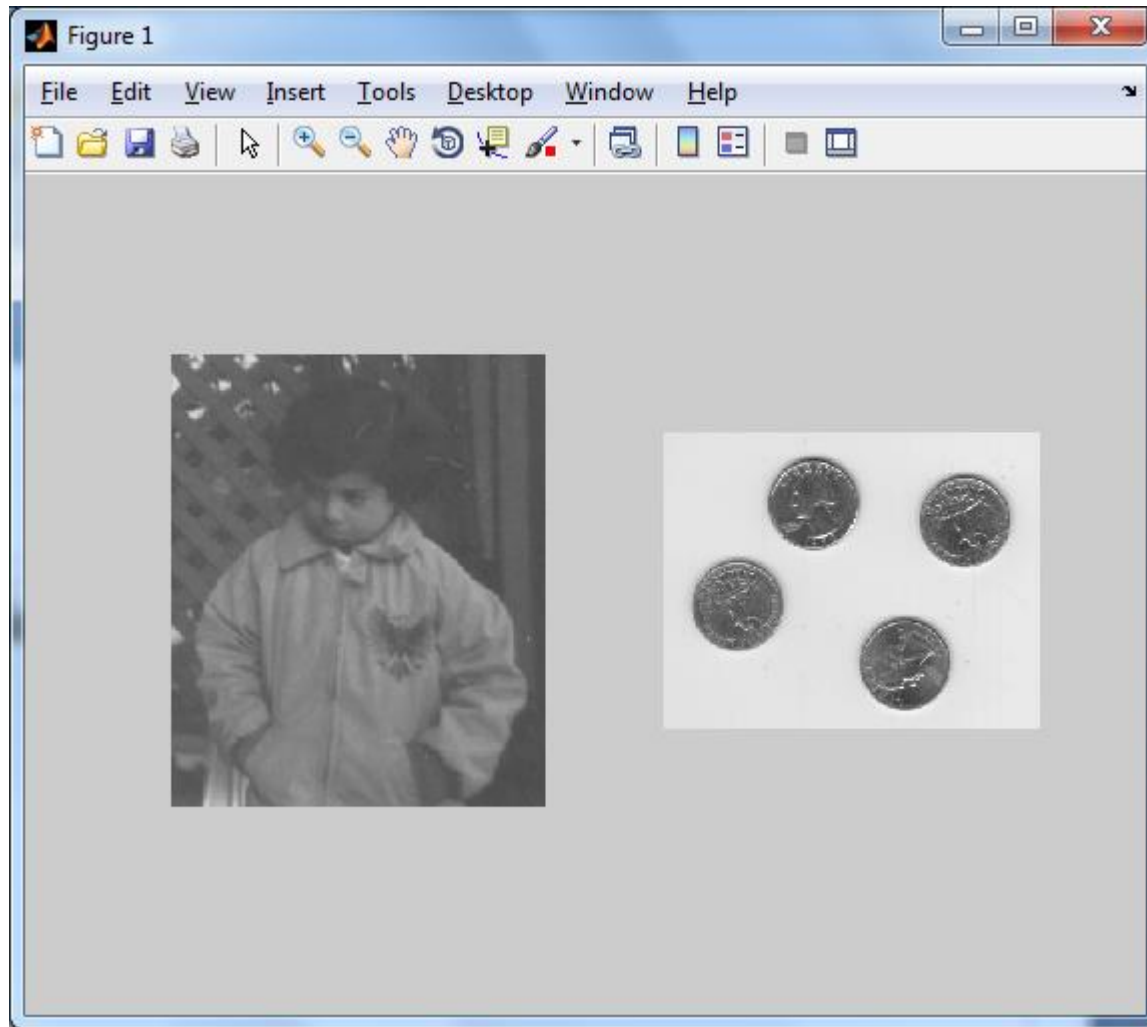
PREPROCESSING AND MATLAB

Intensity transformation functions

Display Multiple Images in the Same Figure

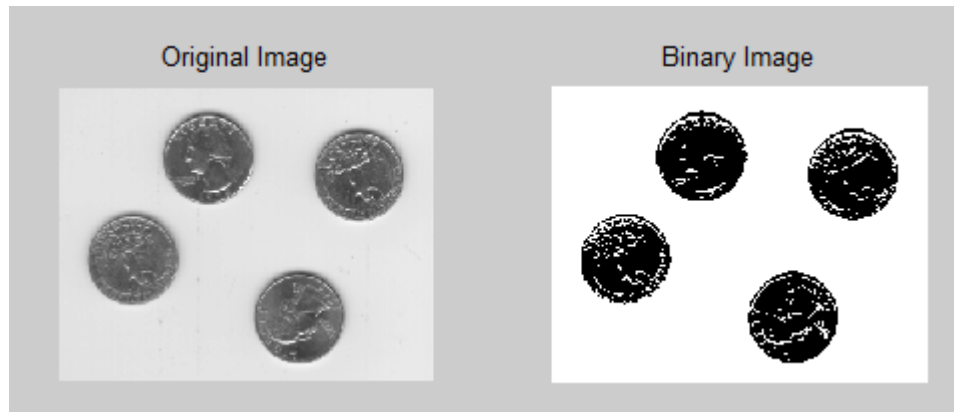
- You can use the **imshow** function with the **subplot** function to display multiple images in a single figure window.
- **subplot(m,n,p)**
- This syntax divides the figure into an m-by-n matrix of display regions and makes the pth display region active.
- **>>c=imread('pout.tif');**
- **>>c2=imread('eight.tif');**
- **>> subplot(1,2,1), imshow(c)**
- **>> subplot(1,2,2), imshow(c2)**

Display Multiple Images in the Same Figure



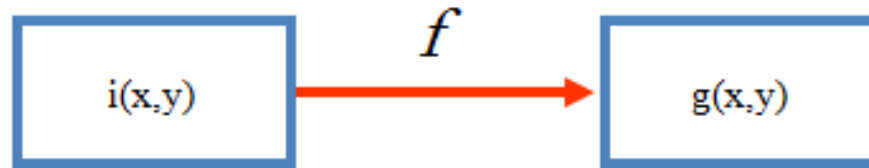
Binary images

- `>>c2=imread('eight.tif');`
- **Convert to binary image**
- `>> bw=im2bw(c2);`
- **Display original image and binary image for comparison.**
- `>> subplot(1,2,1), imshow(c2), title('Original Image');`
- `>> subplot(1,2,2), imshow(bw), title('Binary Image')`



Intensity transformation functions

- intensity transformation function f transforms the given image $i(x, y)$ into another image $g(x, y)$,
- where $g(x, y) = f(i(x, y))$

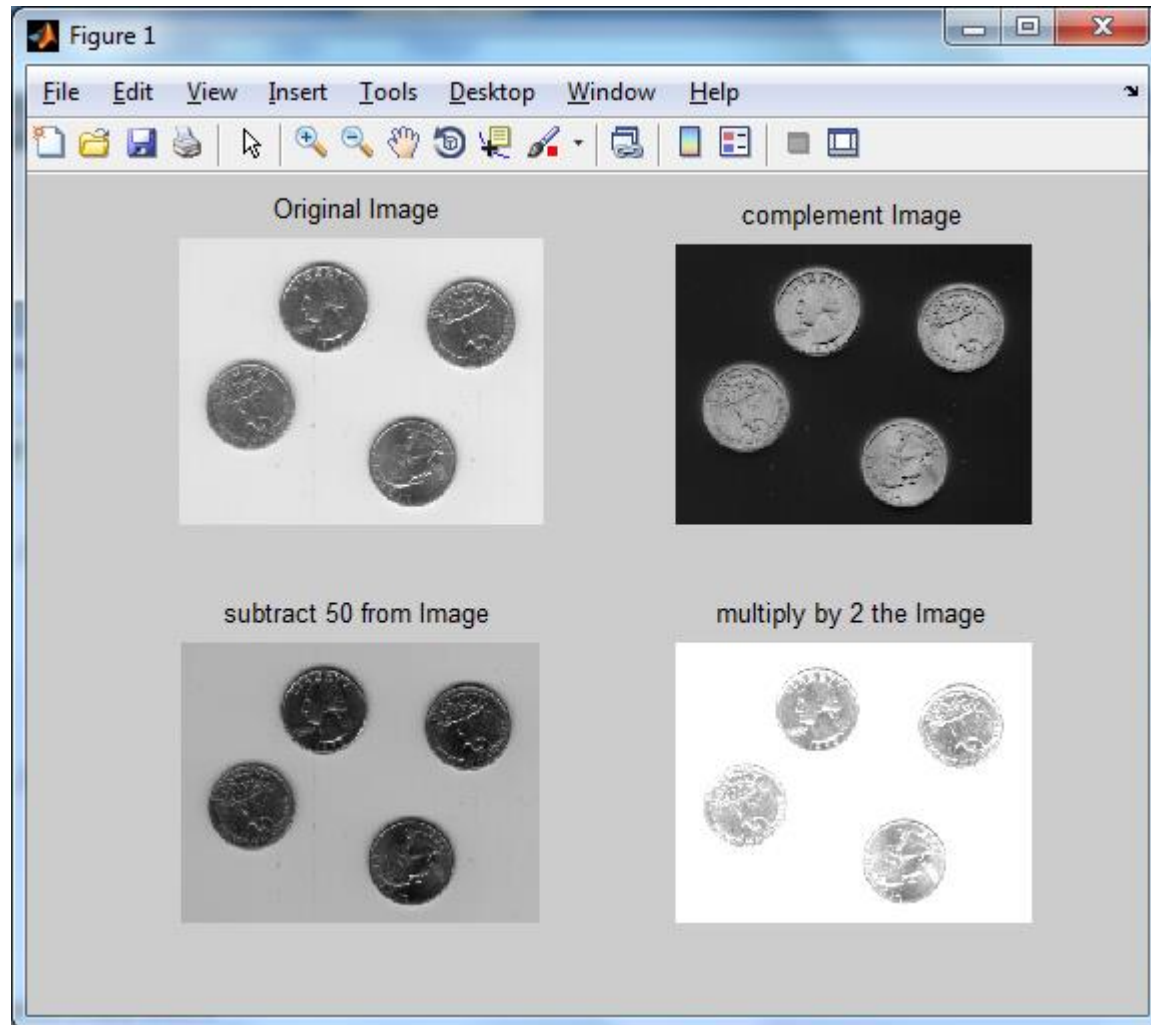


- Image Negatives (complement): $f(x) = 1 - x$
- Arithmetic operations:
- Add **imadd**
- Subtract **imsubtract**
- Multiply **immultiply**
- Divide **imdivide**

Arithmetic operations

- `>>c2=imread('eight.tif');`
- `>> subplot(2,2,1), imshow(c2),title('Original Image');`
- `>> subplot(2,2,2),
imshow(imcomplement(c2)),title('complement
Image');`
- `>> subplot(2,2,3),
imshow(imsubtract(c2,50)),title('subtract 50 from
Image');`
- `>> subplot(2,2,4),
imshow(immultiply(c2,2)),title('multiply by 2 the
Image')`

Arithmetic operations

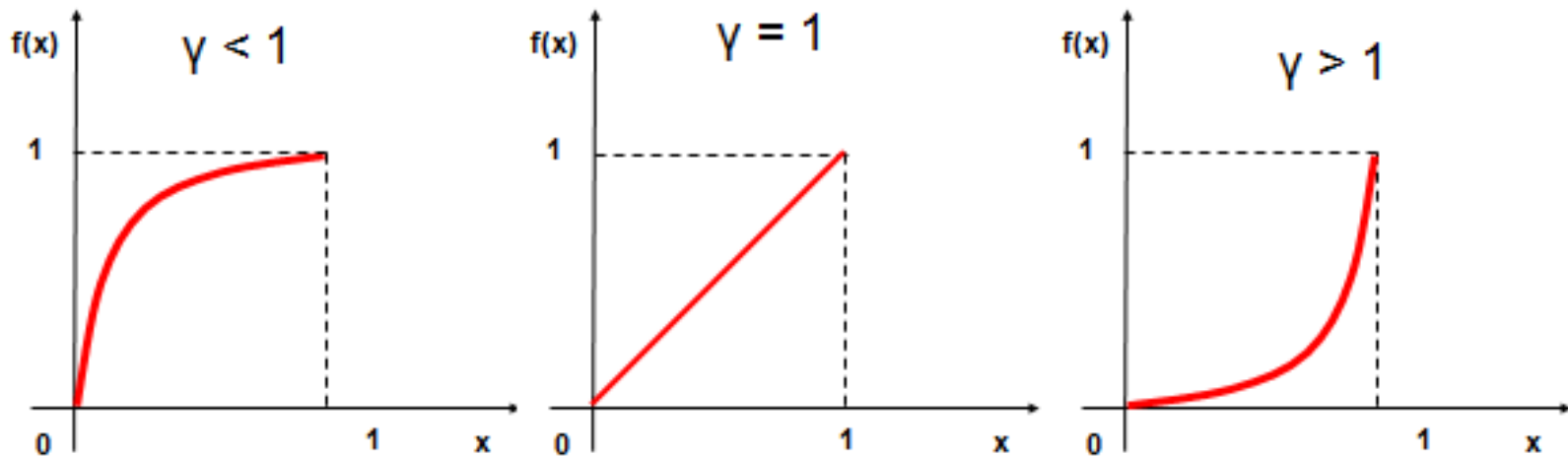


Gamma correction

- The gamma correction is an intensity transformation function in the form

$$f(x) = x^\gamma$$

- where γ is a constant.



Intensity transformation functions

gamma function

Function **imadjust**

`g = imadjust( f, [low_in high_in ], [low_out high_out ], gamma);`

This function maps image `f` to new image `g`

All inputs, other than `f`, between 0 and 1

Using empty `[ ]` result in default `[0 1]`

If `high_out` is less than `low_out` the out intensity is reversed

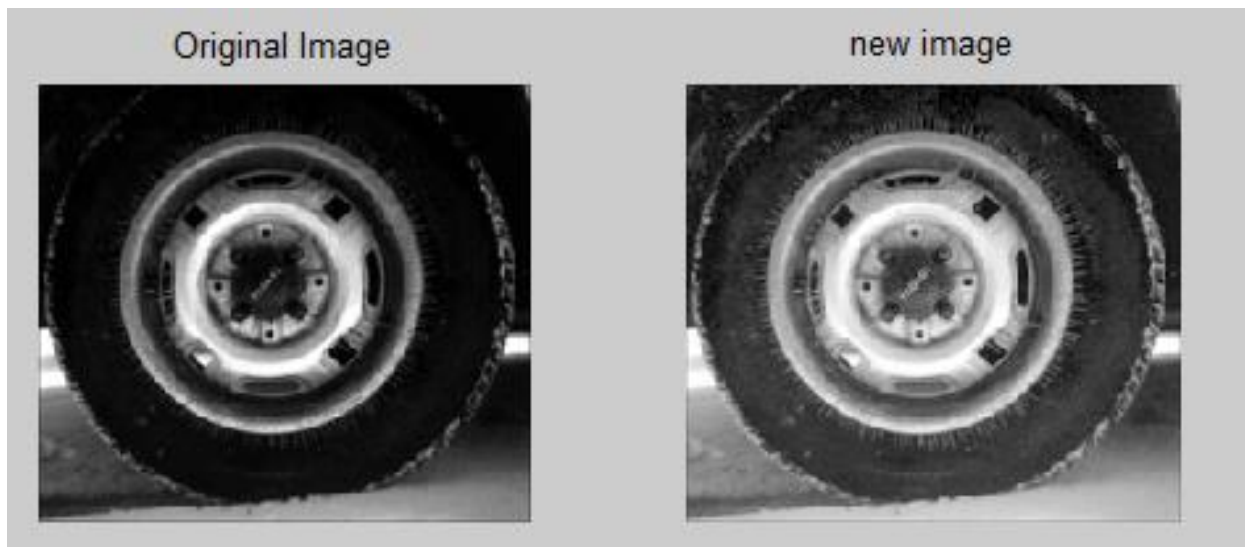
If $\gamma < 1$ the image is weighted toward higher (brighter) output values.

If $\gamma > 1$ the image is weighted toward lower (darker) output values.

Intensity transformation functions

gamma function

- `>> t=imread('tire.tif');`
- **Gamma=0.5**
- `>> th=imadjust(t,[ ],[ ],0.5);`
- `>> subplot(1,2,1),imshow(t),title('Original Image');`
- `>> subplot(1,2,2),imshow(th),title('new image')`



Intensity transformation functions

Negative image

If `high_out` is less than `low_out` the out intensity is reversed

```
>> c2=imread('eight.tif');  
>> g1=imadjust(c2,[0 1],[1 0]);  
>> imshow(c2),figure, imshow(g1)
```



Intensity transformation functions

gamma function

```
>> c2=imread('eight.tif');
```

```
Gamma=2
```

```
>> g2=imadjust(c2,[ ],[ ],2);
```

```
Gamma=0.5
```

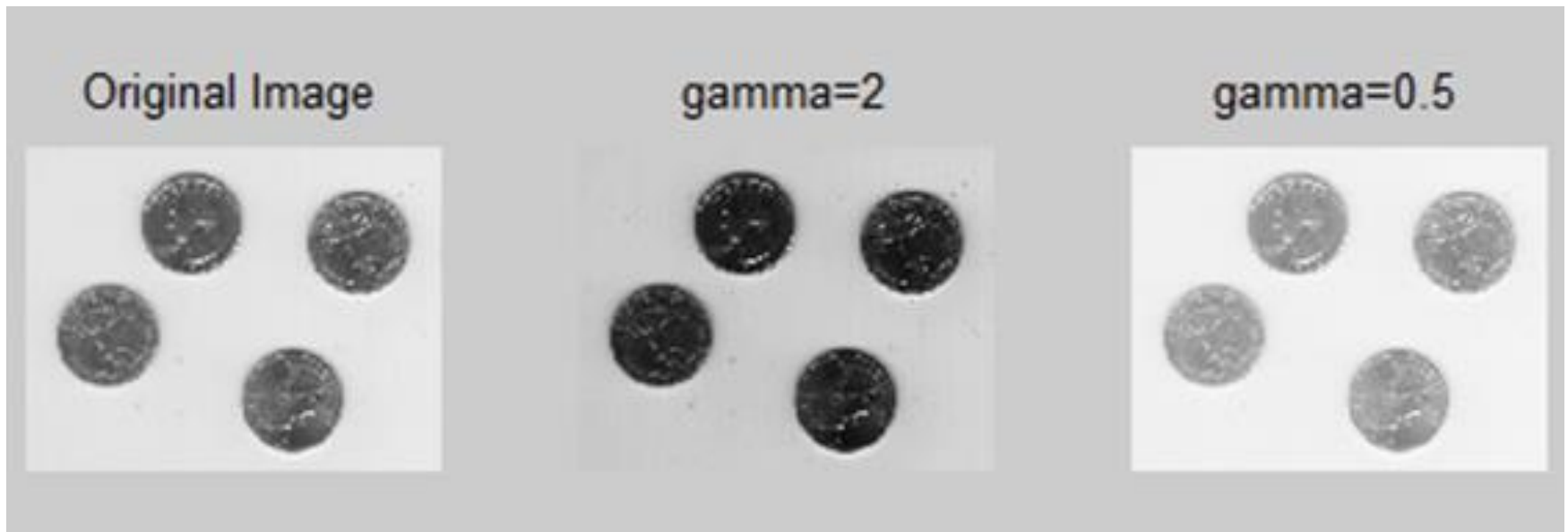
```
>> g3=imadjust(c2,[ ],[ ],0.5);
```

```
>> subplot(1,3,1), imshow(c2), title('Original Image');
```

```
>> subplot(1,3,2), imshow(g2), title('gamma=2');
```

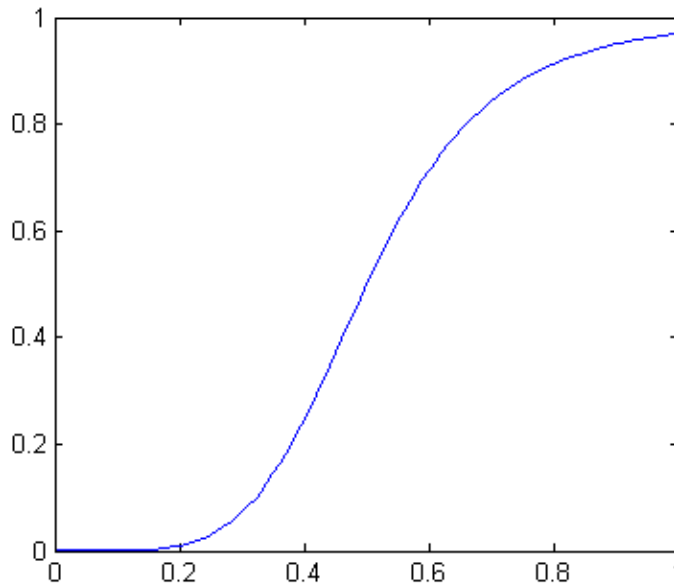
```
>> subplot(1,3,3), imshow(g3), title('gamma=0.5')
```

gamma function



Contrast-stretching transformations

- The graph of the contrast stretching function for $E = 5$



$$f(x) = \frac{1}{1 + (0.5/x)^E}$$

$$f(x) = \frac{1}{1 + (0.5/x)^5}$$

- Pixel values below 0.5 are pushed nearer to 0.
- Pixel values above 0.5 are pushed nearer to 1.

Contrast-stretching transformations

```
>> x=imread('pout.tif');
```

$$f(x) = \frac{1}{1 + (0.5/x)^E}$$

```
E=2
```

```
>> g=1./(1+(0.5./(double(x)+eps)).^2);
```

Note: dot operations page 19 ($x.^2$, $1./x$, $x.*2$...)

eps= 2.2204e-16

```
>> gs1=mat2gray(g);
```

mat2gray Convert matrix to intensity image.

```
E=5
```

```
>> g1=1./(1+(0.5./(double(c)+eps)).^5);
```

```
>> gs2=mat2gray(g1);
```

```
>> imshow(c),figure,imshow(gs1),figure,imshow(gs2)
```

Contrast-stretching transformations

- Original Image $E=2$ $E=5$

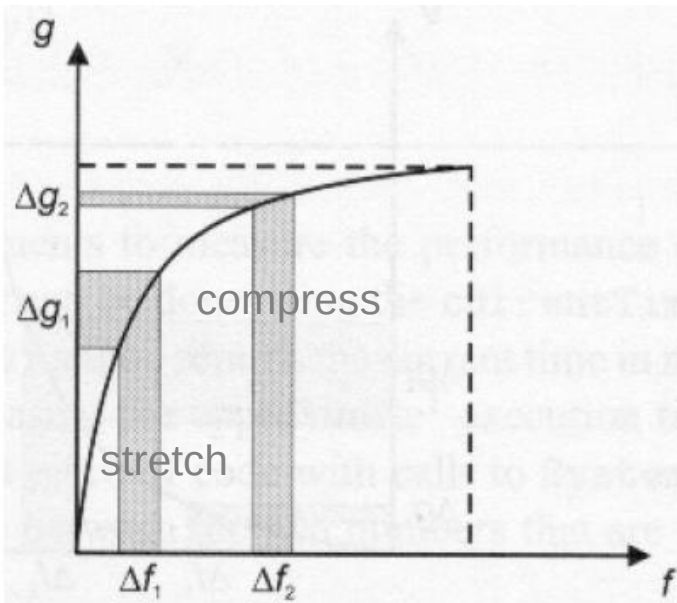


Logarithmic transformation

- Enhance details in the darker regions of an image at the expense of detail in brighter regions.

$$g(f) = c * \log(1 + f)$$

$$>> g = C * \log(1 + \text{double}(f));$$



Logarithmic transformation

- `>> c=imread('pout.tif');`
- `>> g=2*log(1+double(c));`
- `>> gs=mat2gray(g);`
- `>> imshow(c), figure, imshow(gs)`



- `mat2gray` Convert matrix to intensity image.
- The input array `A` can be logical or numeric.
- Example
- -----
- `I = imread('rice.png');`
- `J = filter2(fspecial('sobel'), I);`
- `K = mat2gray(J);`
- `figure, imshow(I), figure, imshow(K)`

Contrast-stretching transformations

```
>> c=imread('pout.tif');
```

```
>> g=1./(1+(0.5./(double(c)+eps)).^2);
```

```
>> gs1=im2uint8(mat2gray(g));
```

mat2gray Convert matrix to intensity image.

im2uint8 Convert image to 8-bit unsigned integers.

```
>> g1=1./(1+(0.5./(double(c)+eps)).^5);
```

```
>> gs2=im2uint8(mat2gray(g1));
```

```
>> imshow(c),figure,imshow(gs1),figure,imshow(gs2)
```

```
>> gs1=mat2gray(g);
```

```
>> figure,imshow(gs1)
```